



Cyberscope

A *TAC Security* Company

Audit Report

Hero Arena Play

May 2026

Repository https://github.com/HaGameFi/HeroArena_contracts

Commit hash 4827a7215d6876ec73d84ab69ce732810595ffcb

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	5
HapToken.sol	5
HapVesting.sol	5
HapTreasury.sol	5
HeroArenaBattle.sol	6
Findings Breakdown	7
Diagnostics	8
RBV - Revocation Breaks Vesting	9
Description	9
Recommendation	10
STI - Shared Token Insolvency	11
Description	11
Recommendation	12
BC - Blacklists Addresses	13
Description	13
Recommendation	13
CSP - Completed Schedules Persist	15
Description	15
Recommendation	16
CCR - Contract Centralization Risk	17
Description	17
Recommendation	17
IPG - Inconsistent Pause Guard	18
Description	18
Recommendation	19
PTAI - Potential Transfer Amount Inconsistency	20
Description	20
Recommendation	20
RWP - Reverting Winner Payout	22
Description	22
Recommendation	23
ST - Stops Transactions	24
Description	24
Recommendation	24

TBB - TransferFrom Blacklist Bypass	26
Description	26
Recommendation	26
TCR - TryPayout Can Revert	27
Description	27
Recommendation	28
ZBB - Zero Bet Bonus	29
Description	29
Recommendation	30
Functions Analysis	31
Summary	35
Disclaimer	36
About Cyberscope	37

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit

11 May 2026

Source Files

Filename	SHA256
HeroArenaBattle.sol	5babca13cd7eec7cabd0c6e1605f2bb0bd 7f3d3b39102abc9a15f9943b134419
HapVesting.sol	d2855aa5cebe613b39ac12afec85b2fbc18 2dbbc94b5611438e6007c8e04cec0
HapTreasury.sol	29a651cf9cc7a2b0e3ec8116dfb615b8fe4 4fa0f152a7e28701802cd27318a55
HapToken.sol	f9fbca5d22edc8e7a852822e219cf49fd827 6f5fd734b5e8389be460d073a137

Overview

The four contracts form the core Hero Arena token and fund-management system.

`HapToken` defines the fixed-supply HAP asset and its compliance controls, `HapVesting` manages scheduled token distribution after TGE, `HapTreasury` holds and disburses long-term project funds through a proposal and timelock flow, and `HeroArenaBattle` handles player battle creation, staking, settlement, protocol fees, and optional bonus rewards. Together, they cover token issuance, vesting allocation, treasury governance, and gameplay-linked token movement.

HapToken.sol

`HapToken` is the fixed-supply ERC20 token for the Hero Arena ecosystem. It mints the full `1,000,000,000 HAP` supply to the initial admin at deployment and does not include any further minting functionality. The contract extends burnable and pausable ERC20 behavior, tracks cumulative burns through `totalBurned`, and uses role-based access control for pausing and blacklist administration. It also includes protected addresses that cannot be blacklisted and an admin-only recovery function for ERC20 tokens accidentally sent to the token contract, excluding HAP itself.

HapVesting.sol

`HapVesting` manages HAP vesting schedules for beneficiaries. Each schedule supports a TGE unlock amount, cliff duration, linear vesting duration, release tracking, and optional revocation for revocable allocations such as team or advisor grants. The contract allows admins to create multiple schedules per beneficiary, lets anyone release vested tokens for a schedule, and provides a batch `releaseAllMine()` flow for beneficiaries. It also tracks aggregate allocated and released amounts, supports revocation with auto-release of vested tokens, and exposes view helpers for frontend schedule and vesting calculations.

HapTreasury.sol

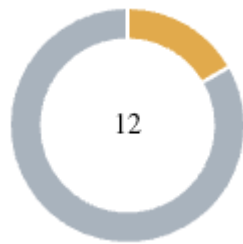
`HapTreasury` is the project treasury manager for native BNB and ERC20 assets. It records received funds, tracks cumulative spending, and disburses assets through a proposal lifecycle: creation by `PROPOSAL_ROLE`, approval by the admin role, a

seven-day timelock, and execution by `EXECUTOR_ROLE` before expiry. It includes cancellation, status, and timing views, plus guardian-controlled emergency pause functionality. When paused, the admin can perform emergency withdrawals, while normal proposal approval and execution are disabled.

HeroArenaBattle.sol

`HeroArenaBattle` manages wager-based battles between registered Hero Arena players. Users can create battles with an allowed bet token, join open or targeted battles, and have results settled by the `LIQUIDATOR_ROLE`. The contract tracks outstanding locked bets, applies optional protocol fees, pays the winner, and can optionally distribute a configured bonus token. It also includes owner-controlled settings for allowed tokens, minimum bet amounts, battle availability, forbidden players, protocol fee configuration, bonus configuration, fee withdrawal, token deposits, and rescue of funds not reserved for active bets or accrued fees.

Findings Breakdown



● Critical	0
● Medium	2
● Minor / Informative	10

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	2	0	0	0
● Minor / Informative	10	0	0	0

Diagnostics

● Critical
 ● Medium
 ● Minor / Informative

Severity	Code	Description	Status
●	RBV	Revocation Breaks Vesting	Unresolved
●	STI	Shared Token Insolvency	Unresolved
●	BC	Blacklists Addresses	Unresolved
●	CSP	Completed Schedules Persist	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	IPG	Inconsistent Pause Guard	Unresolved
●	PTAI	Potential Transfer Amount Inconsistency	Unresolved
●	RWP	Reverting Winner Payout	Unresolved
●	ST	Stops Transactions	Unresolved
●	TBB	TransferFrom Blacklist Bypass	Unresolved
●	TCR	TryPayout Can Revert	Unresolved
●	ZBB	Zero Bet Bonus	Unresolved

RBV - Revocation Breaks Vesting

Criticality	Medium
Location	contracts/HapVesting.sol#L174,L330,L482
Status	Unresolved

Description

`revoke()` can be called before TGE because the constructor only requires `tgeTimestamp` to be in the future. When a schedule is revoked before TGE, `revokedAt` is set to the current block timestamp, which is less than `tgeTimestamp`.

After TGE, `_computeVestedAtTimestamp()` first passes the `timestamp < tgeTimestamp` check, then replaces `effectiveTime` with the earlier `revokedAt`. The subtraction `effectiveTime - tgeTimestamp` then underflows and reverts, breaking vested/releasable calculations for the revoked schedule.

```
require(_tgeTimestamp > block.timestamp + 1 hours, "TGE must be in future");

function revoke(uint256 scheduleId) external onlyRole(VESTING_ADMIN_ROLE) nonReentrant returns (uint256 forfeited) {
    ...
    uint256 vestedSoFar = _computeVestedAmount(schedule);
    ...
    schedule.revoked = true;
    schedule.revokedAt = uint64(block.timestamp);
}

function _computeVestedAtTimestamp(VestingSchedule memory schedule, uint64 timestamp) internal view returns (uint256) {
    if (timestamp < tgeTimestamp) {
        return 0;
    }

    uint64 effectiveTime = timestamp;
    if (schedule.revoked && timestamp > schedule.revokedAt) {
        effectiveTime = schedule.revokedAt;
    }

    uint64 timeElapsed = effectiveTime - tgeTimestamp;
    ...
}
```

```
}
```

Recommendation

Handle revoked schedules whose revocation time is before TGE as a zero-vested case, or ensure the effective vesting timestamp is never earlier than `tgeTimestamp` before performing elapsed-time arithmetic. The revocation and vesting calculation logic should share the same boundary handling for pre-TGE states.

STI - Shared Token Insolvency

Criticality	Medium
Location	contracts/HeroArenaBattle.sol#L79,L201,L407
Status	Unresolved

Description

`settleBattle()` releases the battle's locked stake from `outstandingBets` and then pays the winner. However, the optional bonus is paid from the same raw contract balance without tracking any reserved bonus funds. If `bonusToken == b.betTokenAddress`, bonus payouts can consume tokens that are needed to cover `outstandingBets` for other active battles.

This can make later battles insolvent: their accounting still records locked bets, but the actual token balance may no longer be sufficient to pay winners or refunds.

`rescueExtraTokens()` protects `outstandingBets` and protocol fees, but bonus payouts bypass that reserved-balance check.

```
uint256 totalReward = b.betAmount * 2;

outstandingBets[b.betTokenAddress] -= totalReward;

_payout(b.betTokenAddress, _winner, totalReward);

address _bonusToken = bonusToken;
uint256 _bonusAmount = bonusAmount;
if (_bonusAmount > 0 && _bonusToken != address(0)) {
    bool sent = _tryPayout(_bonusToken, _winner, _bonusAmount);
    emit BonusPayout(_battleId, _winner, _bonusToken, _bonusAmount,
sent);
}

function rescueExtraTokens(address _token, address _to, uint256 _amount)
    external
    onlyOwner
    nonReentrant
{
    ...
    uint256 reserved = outstandingBets[_token] +
accruedProtocolFees[_token];
    uint256 available = totalBalance - reserved;
    require(_amount <= available, "Amount exceeds rescuable balance");
    ...
}
```

Recommendation

Track bonus obligations as reserved funds, especially when the bonus token can also be used as a bet token. Bonus payouts should only be made from unreserved balances after accounting for all active battle stakes, protocol fees, and already-promised bonus amounts.

BC - Blacklists Addresses

Criticality	Minor / Informative
Location	contracts/HapToken.sol#L131
Status	Unresolved

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `blacklistAddress` function.

```
function blacklist(address account) external onlyRole(BLACKLIST_ROLE) {  
    if (account == address(0)) revert CannotBlacklistZeroAddress();  
    if (protectedFromBlacklist[account]) revert  
    CannotBlacklistProtectedAddress(account);  
    if (blacklisted[account]) revert AddressAlreadyBlacklisted(account);  
  
    blacklisted[account] = true;  
    emit AddressBlacklisted(account, msg.sender);  
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

CSP - Completed Schedules Persist

Criticality	Minor / Informative
Location	contracts/HapVesting.sol#L38,L203,L276,L330
Status	Unresolved

Description

`beneficiarySchedules` is append-only and is used to enforce `MAX_SCHEDULES_PER_BENEFICIARY`. When a schedule is fully released or revoked, its ID remains in the beneficiary's schedule list and still counts toward the per-beneficiary limit. After an address accumulates 50 historical schedules, `createVestingSchedule()` will always revert for that beneficiary, even if all previous schedules are completed or revoked.

This can permanently prevent future vesting allocations for active beneficiaries that receive repeated campaign, reward, or operational schedules over time. The cap is intended to bound `releaseAllMine()` gas usage, but without cleanup or active-schedule accounting it also limits the lifetime number of schedules per address.


```
uint256 public constant MAX_SCHEDULES_PER_BENEFICIARY = 50;

function createVestingSchedule(...) external onlyRole(VESTING_ADMIN_ROLE)
returns (uint256 scheduleId) {
    ...
    if (beneficiarySchedules[beneficiary].length >=
MAX_SCHEDULES_PER_BENEFICIARY) revert TooManySchedules();
    ...
    beneficiarySchedules[beneficiary].push(scheduleId);
}

function release(uint256 scheduleId) external nonReentrant {
    ...
    schedule.released += releasable;
    totalReleased += releasable;
    ...
}

function revoke(uint256 scheduleId) external onlyRole(VESTING_ADMIN_ROLE)
nonReentrant returns (uint256 forfeited) {
    ...
    schedule.revoked = true;
    schedule.revokedAt = uint64(block.timestamp);
    ...
}
```

Recommendation

Track active schedules separately from historical schedules, or remove completed and revoked schedules from the beneficiary's active schedule index. The per-beneficiary cap should apply only to schedules that still need to be processed by batch release logic, while preserving historical schedule data for accounting and frontend visibility.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	contracts/HapTreasury.sol contracts/HapToken.sol contracts/HapVesting.sol contracts/HeroArenaBattle.sol
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

IPG - Inconsistent Pause Guard

Criticality	Minor / Informative
Location	contracts/HapTreasury.sol#L168,L189
Status	Unresolved

Description

`receiveFunds()` and `depositBNB()` are guarded with `whenNotPaused`, but the native `receive()` function accepts direct BNB transfers even while the treasury is paused. This makes pause behavior inconsistent: during an emergency pause, explicit deposits are blocked, but direct native transfers can still enter the contract and update `totalReceived`.

If the pause is intended to stop treasury operations during emergencies, allowing direct BNB deposits during the paused state may interfere with accounting, monitoring, or incident response assumptions.

```
function receiveFunds(address token, uint256 amount, string calldata
source) external nonReentrant whenNotPaused {
    if (amount == 0) revert InvalidAmount();

    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    totalReceived[token] += amount;

    emit FundsReceived(token, msg.sender, amount, source);
}

function depositBNB(string calldata source) external payable
whenNotPaused {
    if (msg.value == 0) revert InvalidAmount();
    totalReceived[address(0)] += msg.value;
    emit FundsReceived(address(0), msg.sender, msg.value, source);
}

receive() external payable {
    if (msg.value > 0) {
        totalReceived[address(0)] += msg.value;
        emit FundsReceived(address(0), msg.sender, msg.value,
"DIRECT_TRANSFER");
    }
}
```

```
}
```

Recommendation

Apply the same pause policy consistently to direct native transfers. If deposits should be blocked during emergencies, guard `receive()` with the paused state; if direct transfers must remain accepted, document that exception and consider separating emergency-disabled accounting from unavoidable native-token receipts.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	contracts/HapTreasury.sol#L168
Status	Unresolved

Description

The `transfer()` and `transferFrom()` functions are used to transfer a specified amount of tokens to an address. The fee or tax is an amount that is charged to the sender of an ERC20 token when tokens are transferred to another address. According to the specification, the transferred amount could potentially be less than the expected amount. This may produce inconsistency between the expected and the actual behavior.

The following example depicts the diversion between the expected and actual amount.

Tax	Amount	Expected	Actual
No Tax	100	100	100
10% Tax	100	100	90

```
function receiveFunds(address token, uint256 amount, string calldata
source) external nonReentrant whenNotPaused {
    if (amount == 0) revert InvalidAmount();

    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    totalReceived[token] += amount;

    emit FundsReceived(token, msg.sender, amount, source);
}
```

Recommendation

The team is advised to take into consideration the actual amount that has been transferred instead of the expected.

It is important to note that an ERC20 transfer tax is not a standard feature of the ERC20 specification, and it is not universally implemented by all ERC20 contracts. Therefore, the contract could produce the actual amount by calculating the difference between the transfer call.

```
Actual Transferred Amount = Balance After Transfer - Balance  
Before Transfer
```

RWP - Reverting Winner Payout

Criticality	Minor / Informative
Location	contracts/HeroArenaBattle.sol#L201,L454
Status	Unresolved

Description

`settleBattle()` marks the battle as ended and then pushes the winner payout through `__payout()`. For native ETH payouts, `__payout()` reverts if the winner cannot receive ETH, such as when the winner is a contract without a payable receiver or with a reverting fallback. For ERC20 payouts, non-standard or blocked token transfers can also revert.

Because the payout is performed inline, a reverting winner transfer prevents settlement from completing. This can leave a valid battle unable to settle, keeping funds locked and requiring off-chain intervention or contract changes instead of allowing the winner to claim through a safer flow.

```
function settleBattle(uint256 _battleId, address _winner)
    external
    nonReentrant
    onlyRole(LIQUIDATOR_ROLE)
{
    ...
    b.winner = _winner;
    b.isEnded = true;
    ...
    _payout(b.betTokenAddress, _winner, totalReward);
    ...
}

function _payout(address token, address to, uint256 amount) internal {
    if (token == address(0)) {
        (bool ok, ) = to.call{value: amount}("");
        require(ok, "ETH transfer failed");
    } else {
        IERC20(token).safeTransfer(to, amount);
    }
}
```

Recommendation

Use a pull-payment model for mandatory payouts so settlement records the winner's claimable balance instead of requiring the transfer to succeed immediately. Winners should be able to withdraw owed funds separately, and failed transfers should not prevent the battle from being finalized.

ST - Stops Transactions

Criticality	Minor / Informative
Location	contracts/HapToken.sol#L164,L205
Status	Unresolved

Description

The contract owner has the authority to stop the sales for all users. The owner may take advantage of it by calling `pause()`. Additionally, the sales can be stopped by blacklisting the pair contract.

```
function pause() external onlyRole(PAUSER_ROLE) {
    _pause();
}
...
function _update(
    address from,
    address to,
    uint256 value
) internal override(ERC20, ERC20Pausable) {
    // Blacklist check
    if (blacklisted[from]) revert AddressIsBlacklisted(from);
    if (blacklisted[to]) revert AddressIsBlacklisted(to);

    // Track all burns uniformly: burn(), burnFrom(), and
    burnFromRevenue() all pass through here
    if (to == address(0)) {
        totalBurned += value;
    }

    super._update(from, to, value);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

TBB - TransferFrom Blacklist Bypass

Criticality	Minor / Informative
Location	contracts/HapToken.sol#L205
Status	Unresolved

Description

The blacklist enforcement only checks the token `from` and `to` addresses inside `_update()`. As a result, a blacklisted address can still call `transferFrom()` as the spender for any allowance it received before being blacklisted, provided the token owner and recipient are not blacklisted.

This weakens the blacklist mechanism because sanctioned or blocked accounts are still able to move approved funds on behalf of other users. The spender address is `msg.sender` in the inherited ERC20 `transferFrom()` flow, but it is never checked by `HapToken`.

```
function _update(  
    address from,  
    address to,  
    uint256 value  
) internal override(ERC20, ERC20Pausable) {  
    // Blacklist check  
    if (blacklisted[from]) revert AddressIsBlacklisted(from);  
    if (blacklisted[to]) revert AddressIsBlacklisted(to);  
  
    super._update(from, to, value);  
}
```

Recommendation

Ensure blacklist restrictions also apply to the caller/spender in allowance-based transfers. Consider overriding the relevant transfer approval flow so that blacklisted addresses cannot initiate `transferFrom()` operations, while keeping mint, burn, and internal transfer behavior consistent with the intended blacklist policy.

TCR - TryPayout Can Revert

Criticality	Minor / Informative
Location	contracts/HeroArenaBattle.sol#L201,L255,L472
Status	Unresolved

Description

`_tryPayout()` is intended to return `false` instead of reverting when an optional payout fails. However, for ERC20 transfers it decodes any non-empty return data as a `bool`. If a token returns malformed data or a non-boolean value, `abi.decode(ret, (bool))` reverts.

This breaks the intended graceful-failure behavior. Optional bonus payouts, or other flows using `_tryPayout()`, can still revert settlement instead of emitting a failed payout result and allowing the main battle settlement to complete.

```
function settleBattle(uint256 _battleId, address _winner)
    external
    nonReentrant
    onlyRole(LIQUIDATOR_ROLE)
{
    ...
    address _bonusToken = bonusToken;
    uint256 _bonusAmount = bonusAmount;
    if (_bonusAmount > 0 && _bonusToken != address(0)) {
        bool sent = _tryPayout(_bonusToken, _winner, _bonusAmount);
        emit BonusPayout(_battleId, _winner, _bonusToken, _bonusAmount,
sent);
    }
}

function _tryPayout(address token, address to, uint256 amount) internal
returns (bool ok) {
    if (token == address(0)) {
        (ok, ) = to.call{value: amount}("");
    } else {
        bytes memory data =
abi.encodeWithSelector(IERC20.transfer.selector, to, amount);
        (bool success, bytes memory ret) = token.call(data);
        if (!success) return false;
        ok = (ret.length == 0) || abi.decode(ret, (bool));
    }
}
```

Recommendation

Handle ERC20 return data defensively inside `_tryPayout()` so malformed or unexpected return values produce `false` rather than reverting. Optional payout helpers should isolate all token-transfer failures from the settlement path they are meant to protect.

ZBB - Zero Bet Bonus

Criticality	Minor / Informative
Location	contracts/HeroArenaBattle.sol#L149,L249,L322,L432
Status	Unresolved

Description

`createBattle()` accepts `_betAmount` through `_receiveBet()`, which only checks that the amount is greater than or equal to the configured minimum. If the owner configures the relevant minimum bet to zero, users can create and join battles with a zero bet amount.

On settlement, the main reward is zero, but the optional bonus payout is still attempted whenever `bonusToken` and `bonusAmount` are set. This allows registered users to create zero-cost battles and receive the bonus without staking any value, draining the bonus pool through repeated settlements.

```
function createBattle(address _betTokenAddress, uint256 _betAmount,
address _targetAddress)
    external payable nonReentrant onlyRegisteredAndAllowed
{
    ...
    _receiveBet(_betTokenAddress, _betAmount, msg.sender);
    ...
}

function joinExistBattle(uint256 _battleId) external payable nonReentrant
onlyRegisteredAndAllowed {
    ...
    _receiveBet(b.betTokenAddress, b.betAmount, msg.sender);
    ...
}

function settleBattle(uint256 _battleId, address _winner) external
nonReentrant onlyRole(LIQUIDATOR_ROLE) {
    ...
    uint256 totalReward = b.betAmount * 2;
    ...
    _payout(b.betTokenAddress, _winner, totalReward);

    if (_bonusAmount > 0 && _bonusToken != address(0)) {
        bool sent = _tryPayout(_bonusToken, _winner, _bonusAmount);
        emit BonusPayout(_battleId, _winner, _bonusToken, _bonusAmount,
sent);
    }
}

function _receiveBet(address token, uint256 amount, address from)
internal {
    if (token == address(0)) {
        require(amount >= minBetAmount[0], "Bet amount below minimum");
        require(msg.value == amount, "Incorrect ETH amount sent");
    } else {
        require(amount >= minBetAmount[1], "Bet amount below minimum");
        ...
    }
}
```

Recommendation

Require a strictly positive bet amount for battles that are eligible for bonus payouts, or disable bonus distribution for zero-value battles. Minimum bet configuration should preserve the intended economic cost of participation and should not allow bonus rewards to be claimed without locked stake.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
HeroArenaBattle	Implementation	Ownable, AccessControl, ReentrancyGuard		
		Public	✓	Ownable
		External	Payable	-
	createBattle	External	Payable	nonReentrant onlyRegistered AndAllowed
	joinExistBattle	External	Payable	nonReentrant onlyRegistered AndAllowed
	settleBattle	External	✓	nonReentrant onlyRole
	closeBattle	External	✓	nonReentrant onlyRole
	getBattleInfo	External		-
	getBattleCount	External		-
	updateAllowedBetToken	External	✓	onlyOwner
	updateAvailableCreateBattle	External	✓	onlyOwner
	updateForbiddenToPlay	External	✓	onlyOwner
	updateMinimumBetTokenAmount	External	✓	onlyOwner
	updateBonusToken	External	✓	onlyOwner
	setProtocolFee	External	✓	onlyOwner
	setProtocolFeeRecipient	External	✓	onlyOwner

	withdrawProtocolFees	External	✓	onlyOwner nonReentrant
	depositToken	External	✓	onlyOwner
	rescueExtraTokens	External	✓	onlyOwner nonReentrant
	_receiveBet	Internal	✓	
	_payout	Internal	✓	
	_tryPayout	Internal	✓	
HapVesting	Implementation	AccessContr ol, ReentrancyG uard		
		Public	✓	-
	createVestingSchedule	External	✓	onlyRole
	release	External	✓	nonReentrant
	releaseAllMine	External	✓	nonReentrant
	revoke	External	✓	onlyRole nonReentrant
	rescueRevokedTokens	External	✓	onlyRole
	computeReleasableAmount	External		-
	computeVestedAmount	External		-
	computeReleasableForBeneficiary	External		-
	scheduleCount	External		-
	beneficiaryCount	External		-
	getSchedulesOf	External		-
	getSchedule	External		-
	computeVestedAt	External		-

	_computeReleasableAmount	Internal		
	_computeVestedAmount	Internal		
	_computeVestedAtTimestamp	Internal		
HapTreasury	Implementation	AccessContr ol, ReentrancyG uard, Pausable		
		Public	✓	-
	receiveFunds	External	✓	nonReentrant whenNotPause d
	depositBNB	External	Payable	whenNotPause d
		External	Payable	-
	createProposal	External	✓	onlyRole whenNotPause d
	approveProposal	External	✓	onlyRole whenNotPause d
	executeProposal	External	✓	onlyRole nonReentrant whenNotPause d
	cancelProposal	External	✓	-
	emergencyPause	External	✓	onlyRole
	unpause	External	✓	onlyRole
	emergencyWithdraw	External	✓	onlyRole whenPaused nonReentrant
	balanceOf	External		-
	tokenStats	External		-

	proposalStatus	External		-
	timeUntilExecutable	External		-
HapToken	Implementation	ERC20, ERC20Burnable, ERC20Pausable, AccessControl, ReentrancyGuard		
		Public	✓	ERC20
	setProtected	External	✓	onlyRole
	blacklist	External	✓	onlyRole
	unblacklist	External	✓	onlyRole
	pause	External	✓	onlyRole
	unpause	External	✓	onlyRole
	burnFromRevenue	External	✓	nonReentrant
	_update	Internal	✓	
	rescueERC20	External	✓	onlyRole
	burnStats	External		-

Summary

Hero Arena contracts implement a token ecosystem built around the fixed-supply HAP token, scheduled vesting distributions, treasury fund management, and wager-based player battles with optional bonus rewards. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io